



Informed Search & Hill Climbing

Heuristics · Greedy Best-First · A* · Local Search · Hill Climbing

Course No: 0613-401

Course Title: Artificial Intelligence

Ekramul Haque Amin

Lecturer

Department of Artificial Intelligence and Data Science

Dhaka International University (DIU)

Today's Content

- ▶ **Recap: uninformed search and its limitation**
- ▶ **Heuristic functions**
- ▶ **Greedy Best-First Search — step by step**
- ▶ **A* Search — step by step**
- ▶ **Admissible & consistent heuristics; 8-puzzle heuristics**
- ▶ **Local search: Hill Climbing (with Python)**
- ▶ **Problems of hill climbing and how to escape them**

Recap: Why Uninformed Search Is Not Enough

- ▶ Uninformed search (BFS, DFS, UCS, IDS) knows only $g(n)$ — the cost so far.
- ▶ It has no sense of direction: it explores all directions equally.
- ▶ UCS from Dhaka to Chattogram expanded 8 cities, including Gazipur and Narayanganj, which lead away from Chattogram.
- ▶ Idea: give the agent an estimate of the remaining distance → informed (heuristic) search.
- ▶ Informed search usually finds a solution much faster.

The Heuristic Function $h(n)$

- ▶ **$h(n)$ = estimated cost of the cheapest path from n to the goal**
- ▶ **$h(\text{goal}) = 0$**
- ▶ **Comes from problem knowledge, e.g. straight-line distance on a map**
- ▶ **Each algorithm expands the node with the lowest $f(n)$:**
 - **Uniform-Cost: $f(n) = g(n)$ (cost so far)**
 - **Greedy Best-First: $f(n) = h(n)$ (estimate to goal)**
 - **A*: $f(n) = g(n) + h(n)$ (both)**

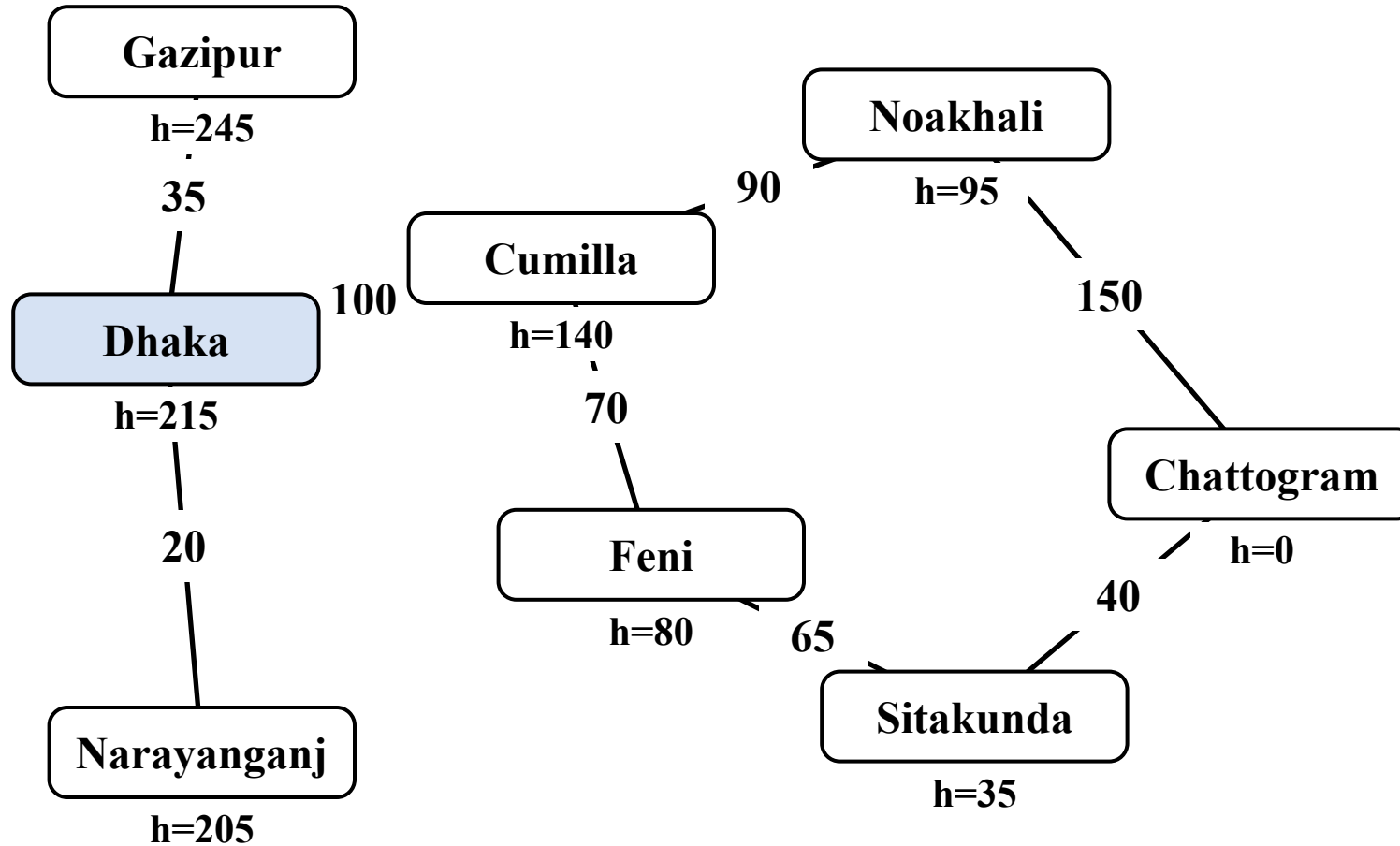
City	$h(n)$ to Chattogram
Dhaka	215
Gazipur	245
Narayanganj	205
Cumilla	140
Noakhali	95
Feni	80
Sitakunda	35
Chattogram	0

h = approximate straight-line distance (km)

Greedy Best-First Search: The Idea

- ▶ Always expand the node that looks closest to the goal: the lowest $h(n)$.
- ▶ Frontier = priority queue ordered by $h(n)$.
- ▶ It ignores the cost already travelled, $g(n)$.
- ▶ How to read the next slides:
 - Yellow = city being expanded now
 - Blue = in the frontier (waiting)
 - Grey = already explored
 - White = not reached yet
 - Table: frontier sorted by h ; the top row is expanded next

Greedy Best-First: Step by Step (0/5)



Start

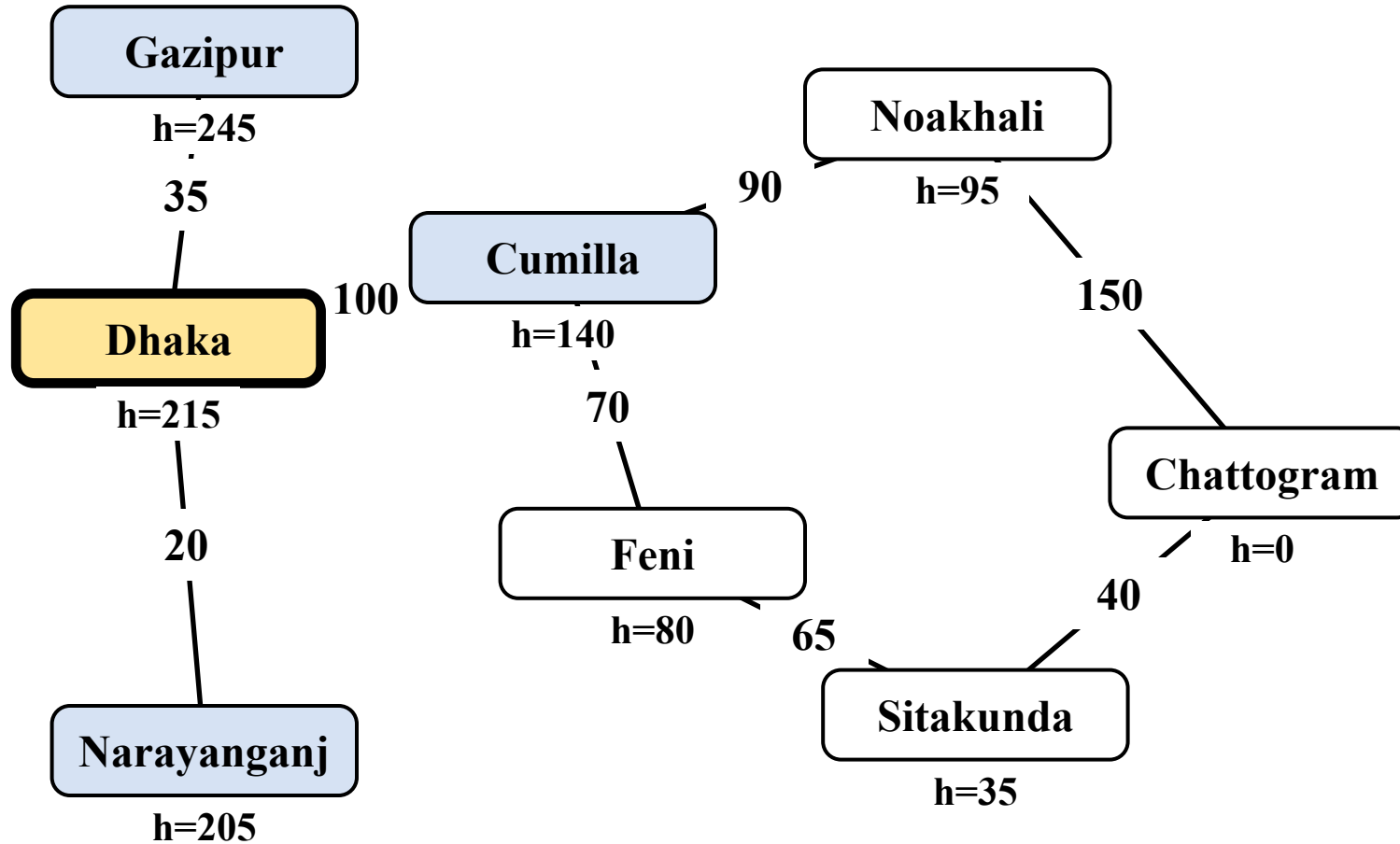
Put the start city Dhaka in the frontier.
Greedy will always remove the city with the LOWEST h.

Frontier (lowest value first):

City	h(n)
Dhaka	215

Explored: —

Greedy Best-First: Step by Step (1/5)



Step 1: expand Dhaka

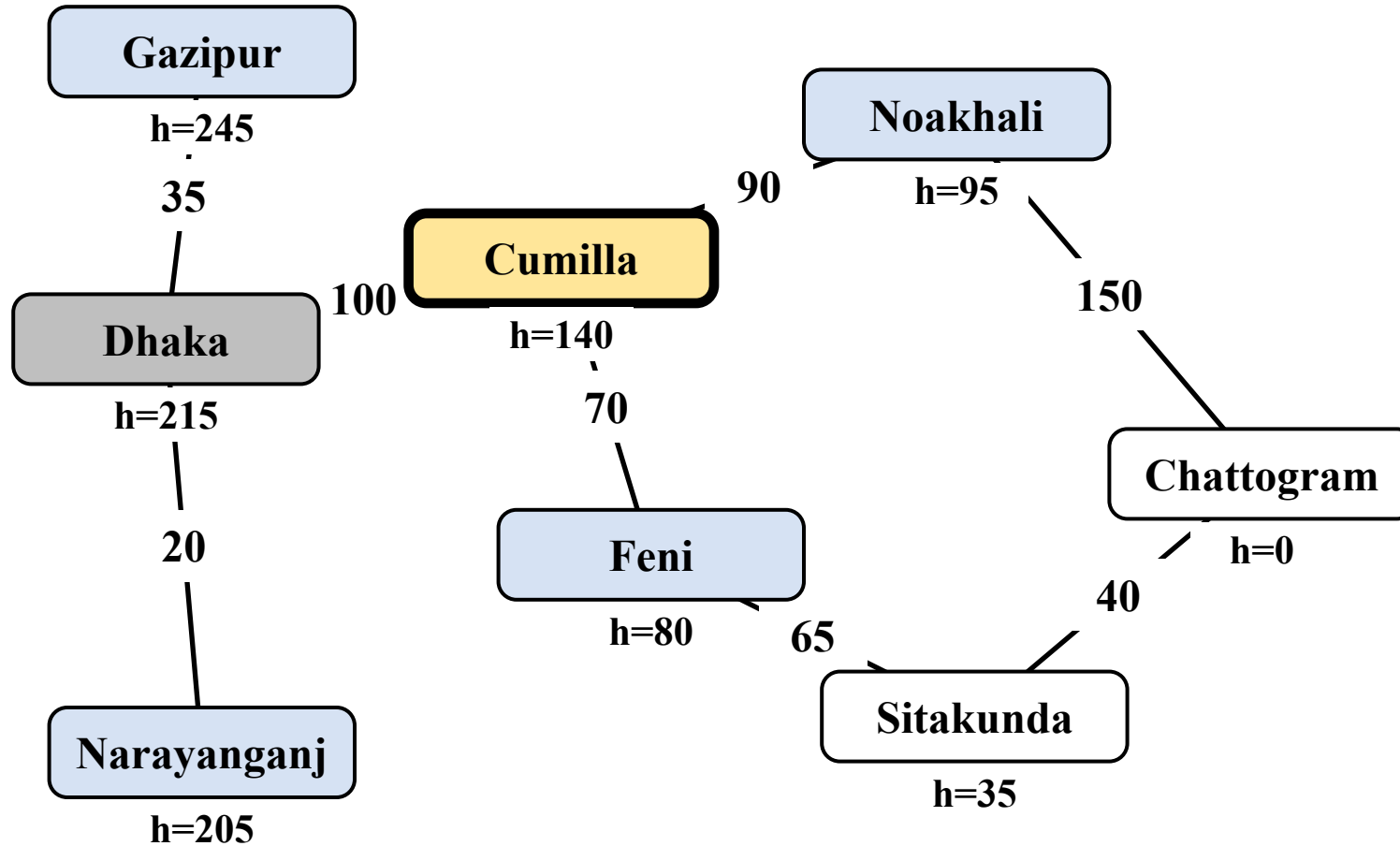
Dhaka is not the goal. Add its neighbours with their h values.

Frontier (lowest value first):

City	h(n)
Cumilla	140
Narayanganj	205
Gazipur	245

Explored: Dhaka

Greedy Best-First: Step by Step (2/5)



Step 2: expand Cumilla (h = 140)

Lowest h in the frontier. Not the goal → add Feni and Noakhali.

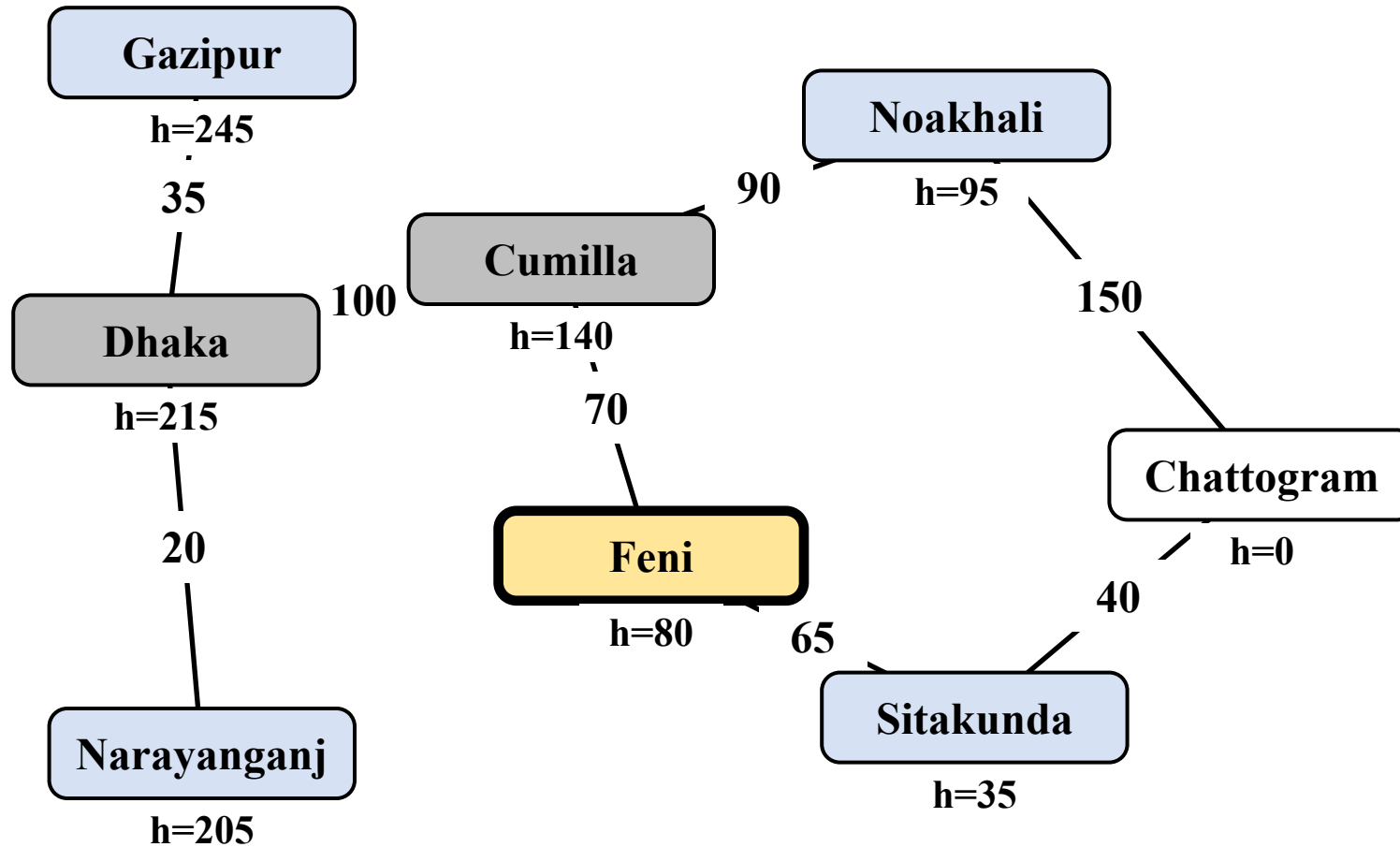
Frontier (lowest value first):

City	h(n)
Feni	80
Noakhali	95
Narayanganj	205
Gazipur	245

Explored: Dhaka, Cumilla

Current
 Frontier
 Explored
 Not reached

Greedy Best-First: Step by Step (3/5)



Current
 Frontier
 Explored
 Not reached

Step 3: expand Feni (h = 80)

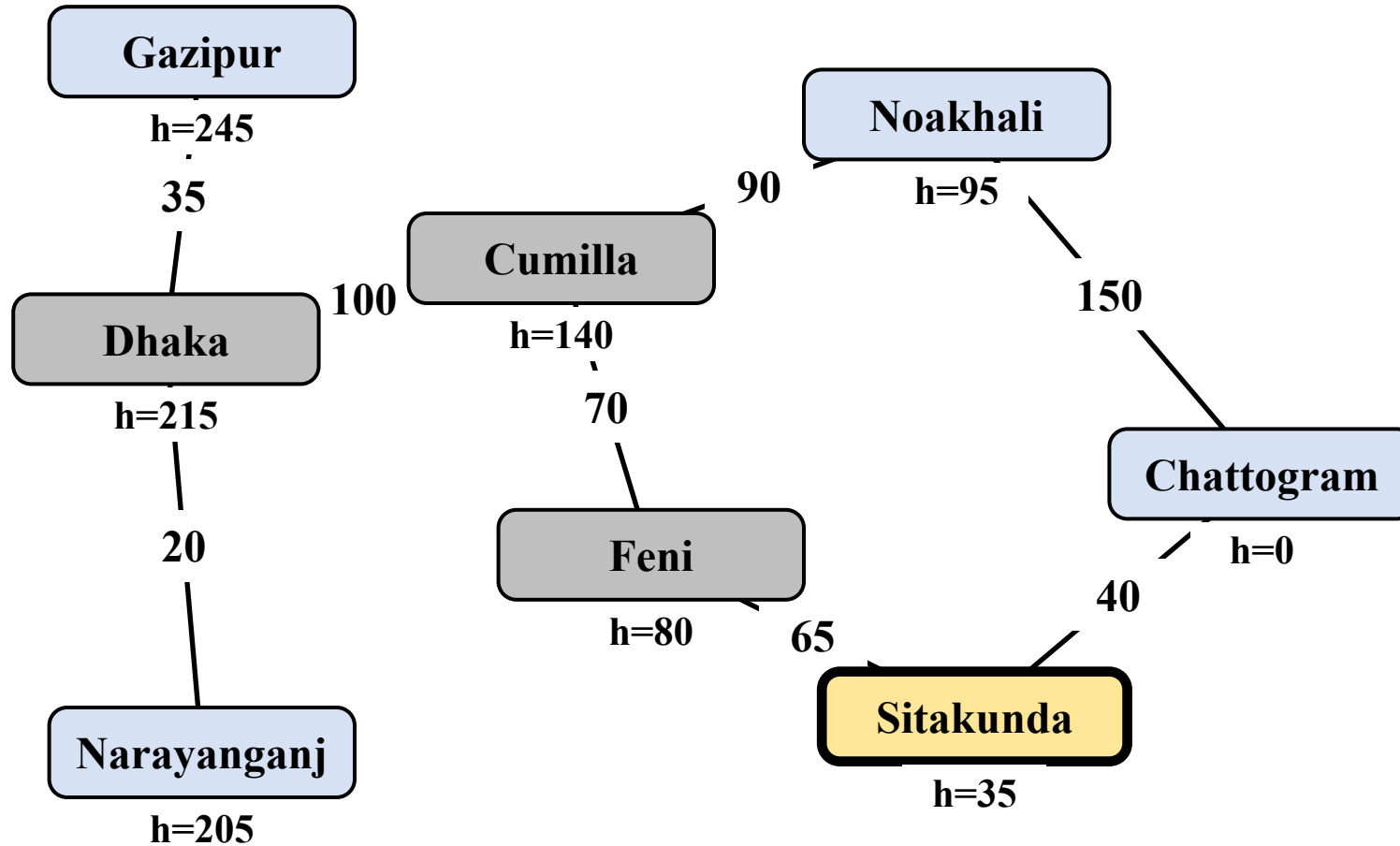
Feni looks closest to Chattogram. Add Sitakunda.

Frontier (lowest value first):

City	h(n)
Sitakunda	35
Noakhali	95
Narayanganj	205
Gazipur	245

Explored: Dhaka, Cumilla, Feni

Greedy Best-First: Step by Step (4/5)



Step 4: expand Sitakunda (h = 35)

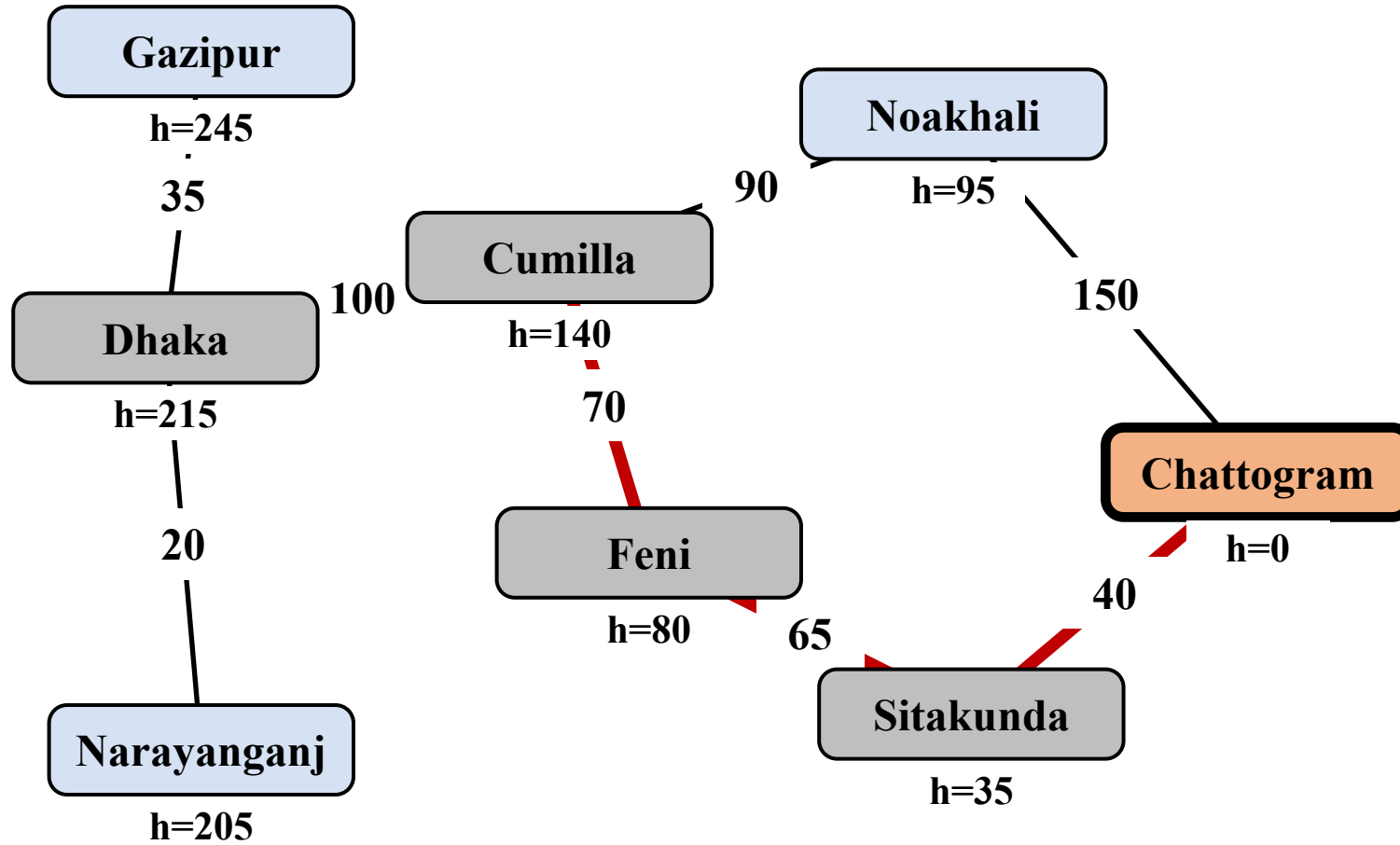
Add Chattogram (h = 0).

Frontier (lowest value first):

City	h(n)
Chattogram	0
Noakhali	95
Narayanganj	205
Gazipur	245

Explored: Dhaka, Cumilla, Feni, Sitakunda

Greedy Best-First: Step by Step (5/5)



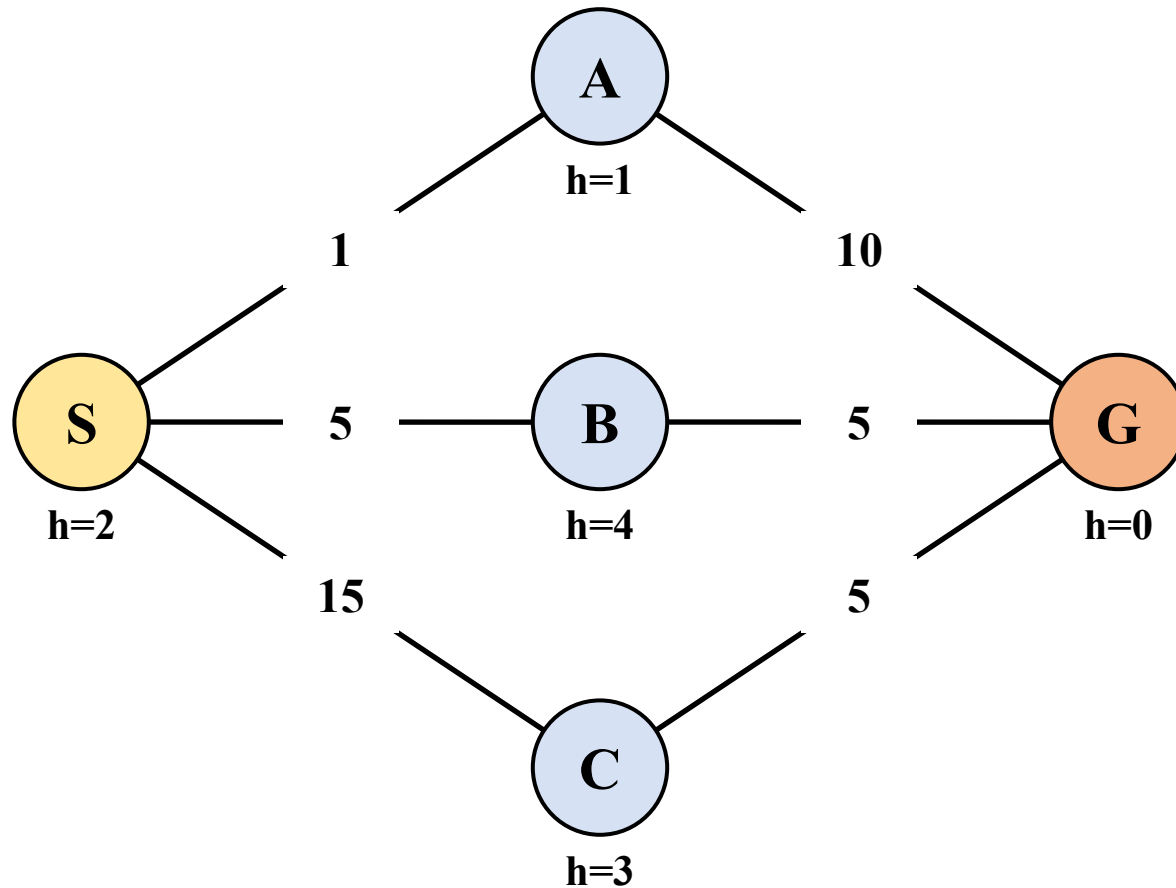
Step 5: Chattogram — GOAL ✓

Path: Dhaka → Cumilla → Feni → Sitakunda → Chattogram. Cost = 100 + 70 + 65 + 40 = 275 km. Only 5 expansions.

Explored: Dhaka, Cumilla, Feni, Sitakunda, Chattogram

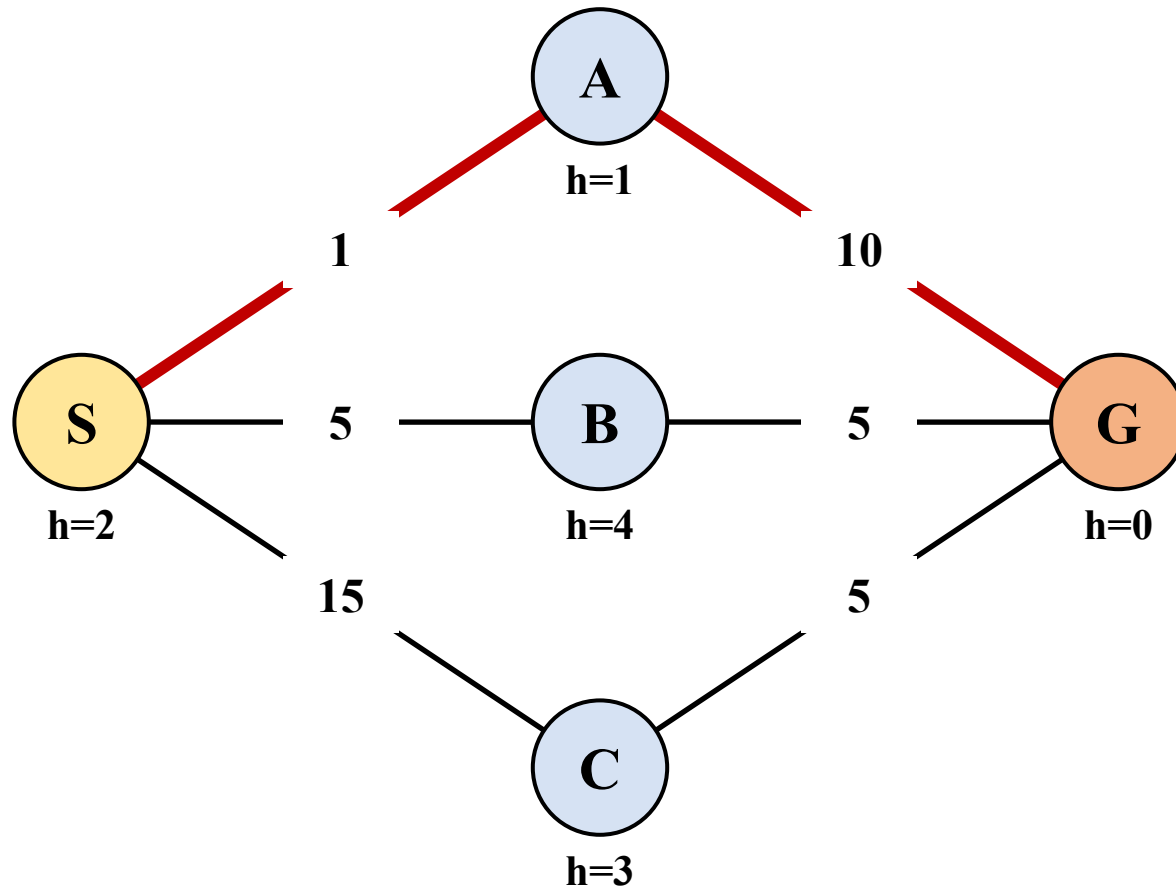
Current
 Frontier
 Explored
 Not reached

Think First: Can Greedy Be Fooled?



- ▶ Start = S, Goal = G. Numbers on edges are costs; h values below nodes.
- ▶ Which path does Greedy return? What is its cost?
- ▶ What is the cheapest path?
- ▶ Discuss with your neighbour for 2 minutes.

Greedy Gets Fooled



Step	Expand	Frontier (h)
1	S	A 1, C 3, B 4
2	A	G 0, C 3, B 4
3	G	Goal ✓

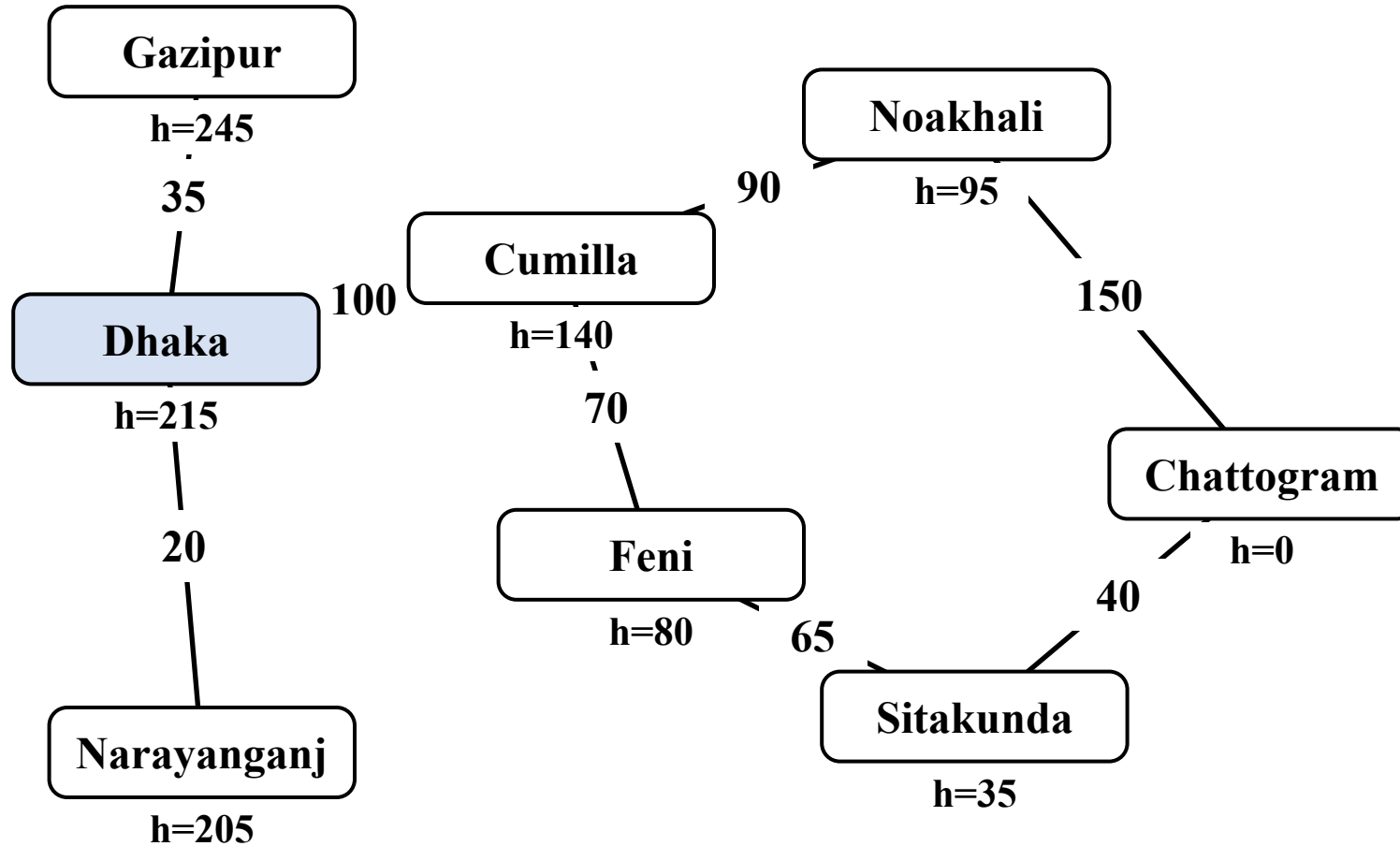
- ▶ Greedy returns $S \rightarrow A \rightarrow G = 11$
- ▶ Cheapest path is $S \rightarrow B \rightarrow G = 10$ ✗
- ▶ Not optimal: it ignores $g(n)$
- ▶ Not complete in tree search (it can loop)

A* Search: The Idea

$$f(n) = g(n) + h(n)$$

- ▶ $g(n)$ = actual cost from start to n (what we already paid)
- ▶ $h(n)$ = estimated cost from n to the goal (what we still expect to pay)
- ▶ $f(n)$ = estimated total cost of the best path through n
- ▶ Expand the node with the lowest $f(n)$; goal test when a node is removed.
- ▶ Same colours and table as before — the table now shows $g + h = f$.

A* Search: Step by Step (0/6)



Start

Frontier holds Dhaka: $g = 0$, $h = 215$, $f = 215$. A* removes the LOWEST $f = g + h$.

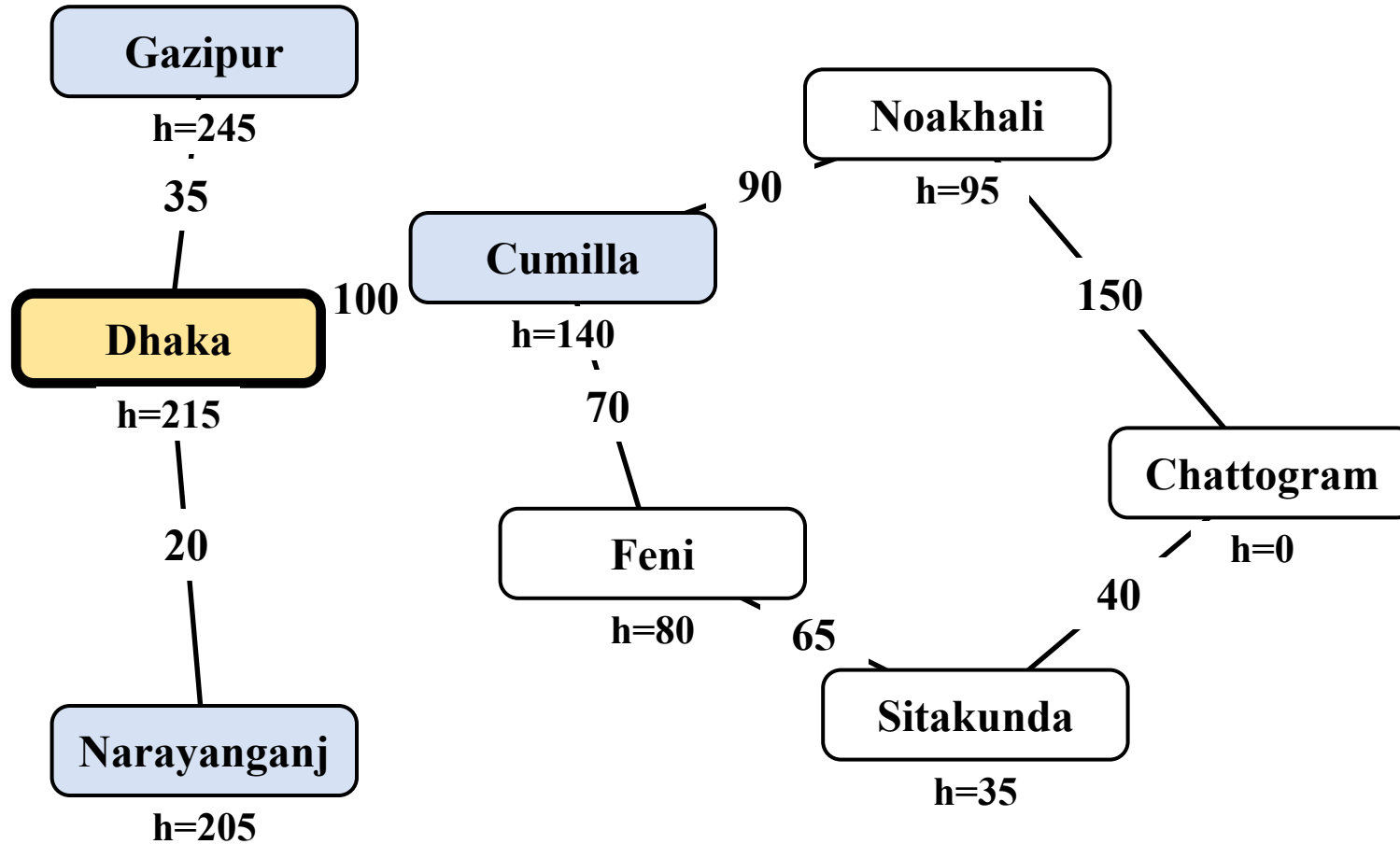
Frontier (lowest value first):

City	$f = g + h$
Dhaka	$0 + 215 = 215$

Explored: —

Current
 Frontier
 Explored
 Not reached

A* Search: Step by Step (1/6)



Step 1: expand Dhaka (f = 215)

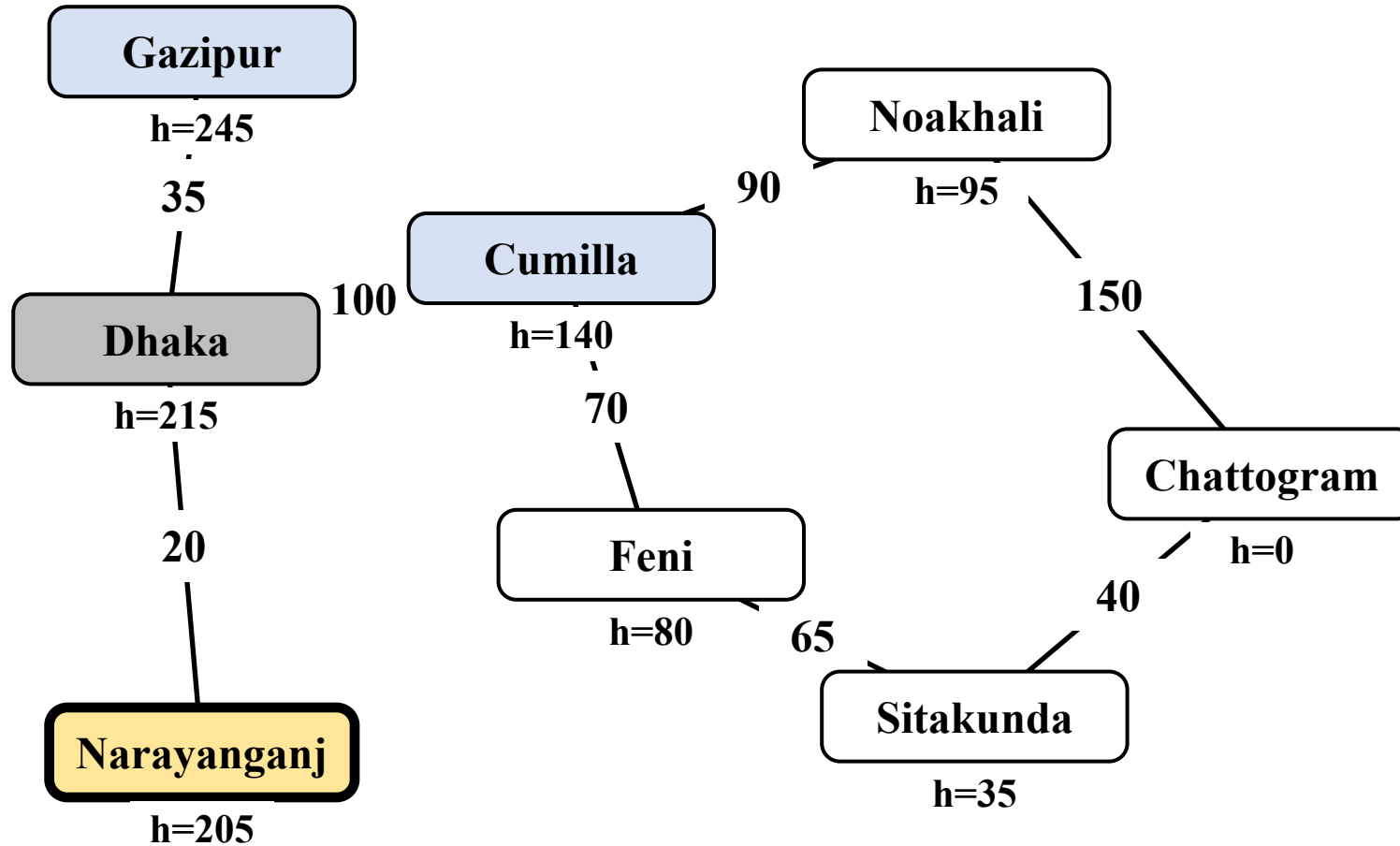
g of each neighbour = road distance from Dhaka.

Frontier (lowest value first):

City	f = g + h
Narayanganj	20 + 205 = 225
Cumilla	100 + 140 = 240
Gazipur	35 + 245 = 280

Explored: Dhaka

A* Search: Step by Step (2/6)



Current
 Frontier
 Explored
 Not reached

Step 2: expand Narayanganj ($f = 225$)

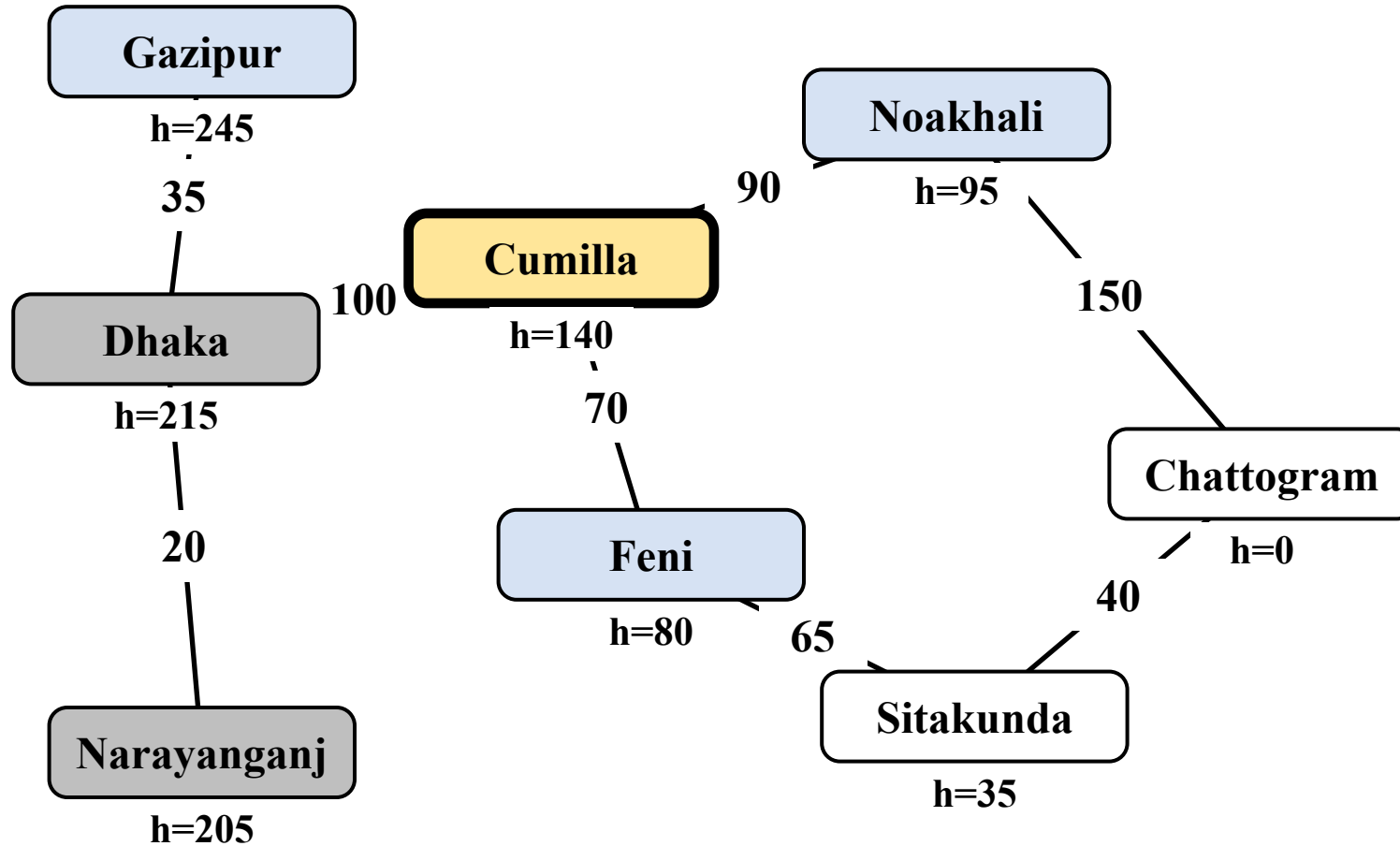
Lowest f , but a dead end: its only neighbour Dhaka is already explored.

Frontier (lowest value first):

City	$f = g + h$
Cumilla	$100 + 140 = 240$
Gazipur	$35 + 245 = 280$

Explored: Dhaka, Narayanganj

A* Search: Step by Step (3/6)



Step 3: expand Cumilla ($f = 240$)

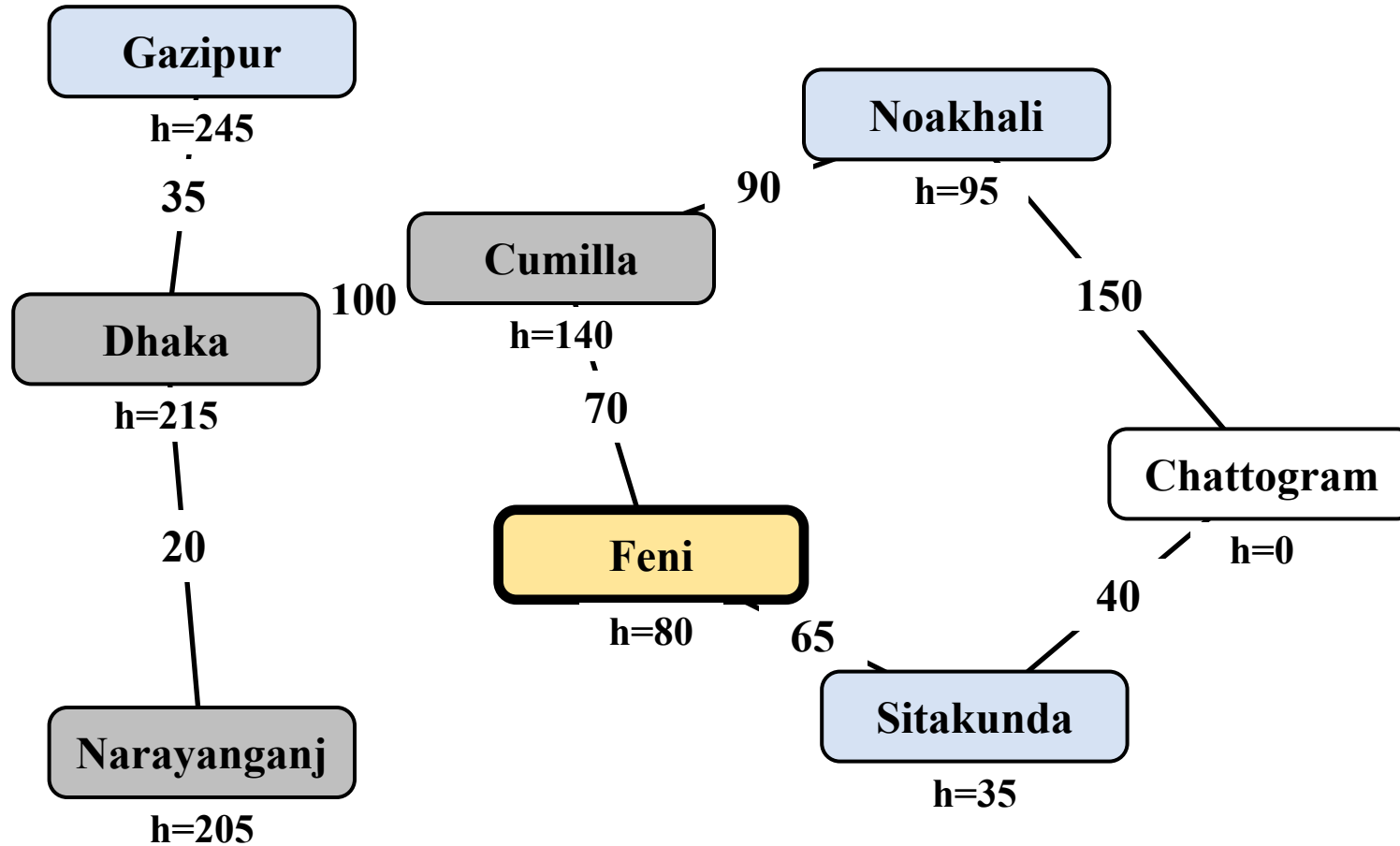
Feni: $g = 100 + 70 = 170$. Noakhali: $g = 100 + 90 = 190$.

Frontier (lowest value first):

City	$f = g + h$
Feni	$170 + 80 = 250$
Gazipur	$35 + 245 = 280$
Noakhali	$190 + 95 = 285$

Explored: Dhaka, Narayanganj, Cumilla

A* Search: Step by Step (4/6)



Step 4: expand Feni (f = 250)

Sitakunda: $g = 170 + 65 = 235$, $f = 235 + 35 = 270$.

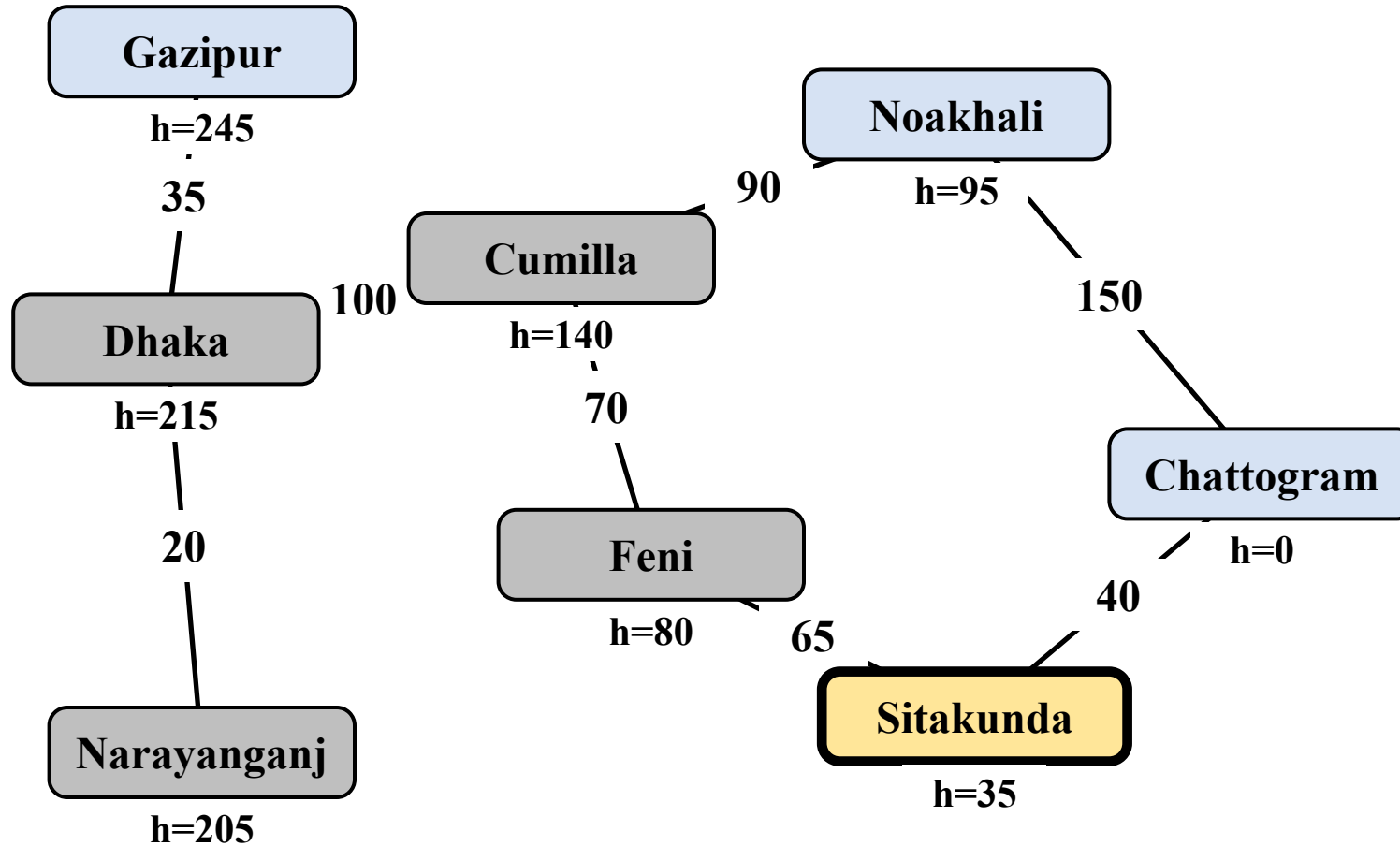
Frontier (lowest value first):

City	$f = g + h$
Sitakunda	$235 + 35 = 270$
Gazipur	$35 + 245 = 280$
Noakhali	$190 + 95 = 285$

Explored: Dhaka, Narayanganj, Cumilla, Feni

Current
 Frontier
 Explored
 Not reached

A* Search: Step by Step (5/6)



Current
 Frontier
 Explored
 Not reached

Step 5: expand Sitakunda (f = 270)

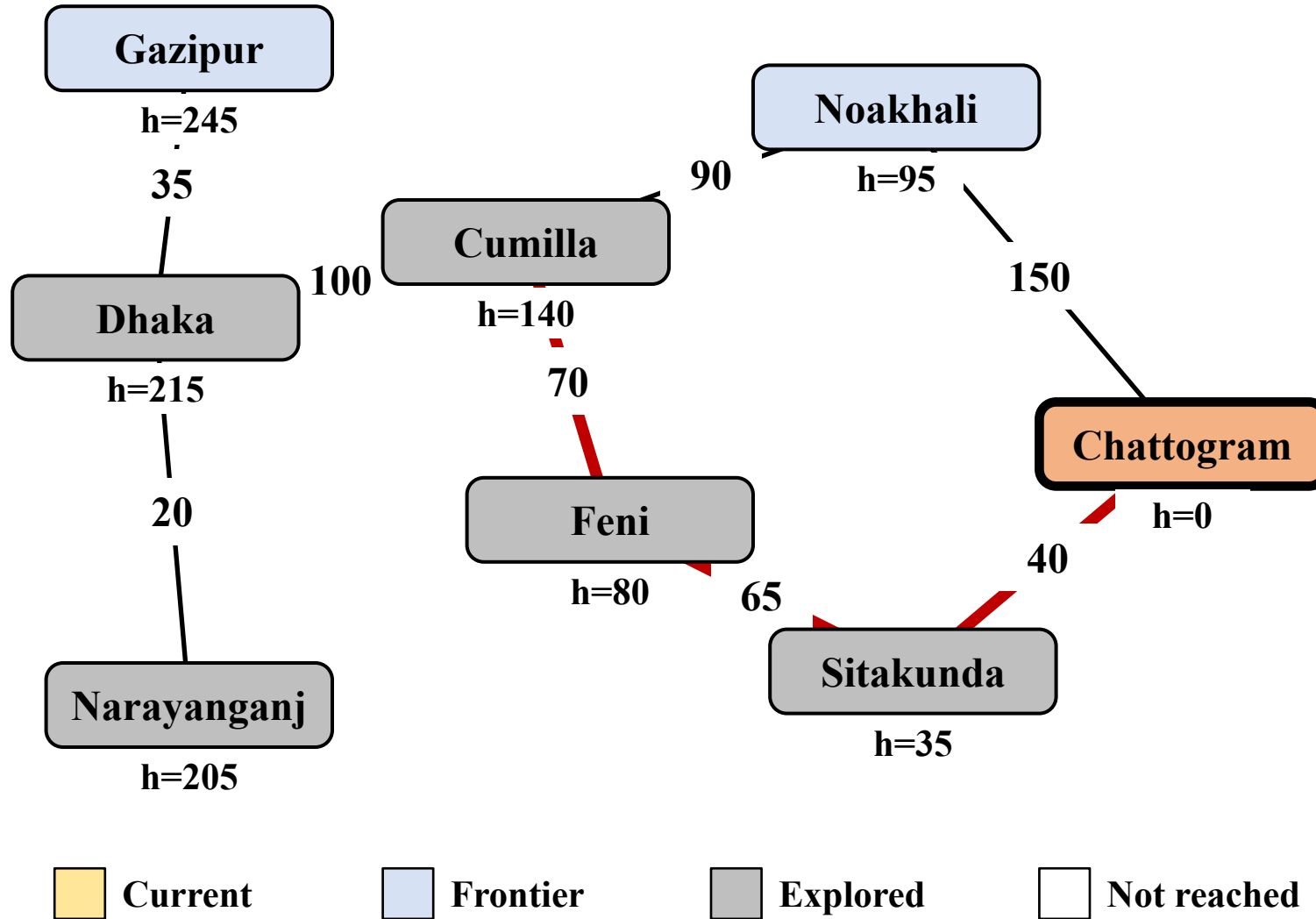
Chattogram: $g = 235 + 40 = 275$, $f = 275 + 0 = 275$. Not tested yet — goal test happens when it is REMOVED.

Frontier (lowest value first):

City	$f = g + h$
Chattogram	$275 + 0 = 275$
Gazipur	$35 + 245 = 280$
Noakhali	$190 + 95 = 285$

Explored: Dhaka, Narayanganj, Cumilla, Feni, Sitakunda

A* Search: Step by Step (6/6)

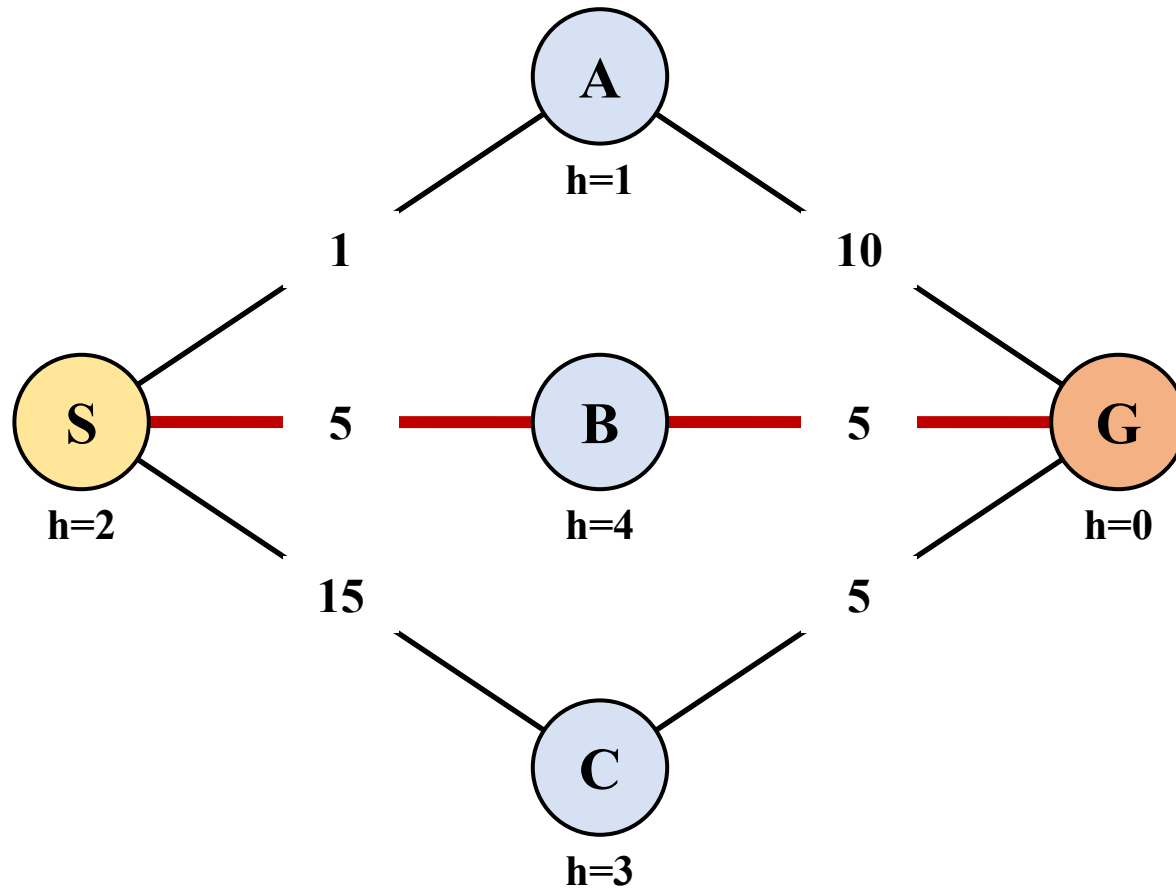


Step 6: Chattogram (f = 275) — GOAL ✓

Lowest f in the frontier, so it is removed and passes the goal test. Optimal path = 275 km with 6 expansions (UCS needed 8).

Explored: Dhaka, Narayanganj, Cumilla, Feni, Sitakunda, Chattogram

A* Is Not Fooled



Step	Expand (f)	Frontier (f = g + h)
1	S 2	A 1+1=2, B 5+4=9, C 15+3=18
2	A 2	B 9, G 11+0=11, C 18
3	B 9	G 10+0=10 (cheaper!), C 18
4	G 10	Goal ✓

► A* returns S → B → G = 10 — optimal ✓

When Is A* Optimal? Admissible Heuristics

- ▶ **Admissible**: h never overestimates: $h(n) \leq h^*(n)$ (true cost)
 - Feni: $h = 80 \leq \text{true } 105$ ✓
- ▶ **Consistent**: $h(n) \leq c(n, n') + h(n')$
 - Cumilla: $140 \leq 70 + 80 = 150$ ✓
- ▶ Consistent $\Rightarrow$ admissible
- ▶ A* is **optimal** with an admissible h (tree search) / consistent h (graph search)
- ▶ Overestimating $h \rightarrow$ A* may return a worse path

City	$h(n)$	$h^*(n)$
Dhaka	215	275
Gazipur	245	310
Narayanganj	205	295
Cumilla	140	175
Noakhali	95	150
Feni	80	105
Sitakunda	35	40
Chattogram	0	0

$h^*(n)$ = true shortest road distance

Heuristics for the 8-Puzzle

2	8	3
1	6	4
7		5

Start

1	2	3
8		4
7	6	5

Goal

- ▶ **h1** = number of misplaced tiles
- ▶ **h2** = Manhattan distance: total horizontal + vertical moves each tile needs
- ▶ Both admissible; **h2 ≥ h1**, so h2 is better (fewer nodes expanded)

Tile	1	2	3	4	5	6	7	8	Total
h1: misplaced?	1	1	0	0	0	1	0	1	4
h2: distance	1	1	0	0	0	1	0	2	5

True optimal solution = 5 moves, so h2 is exact here.

UCS vs Greedy vs A*

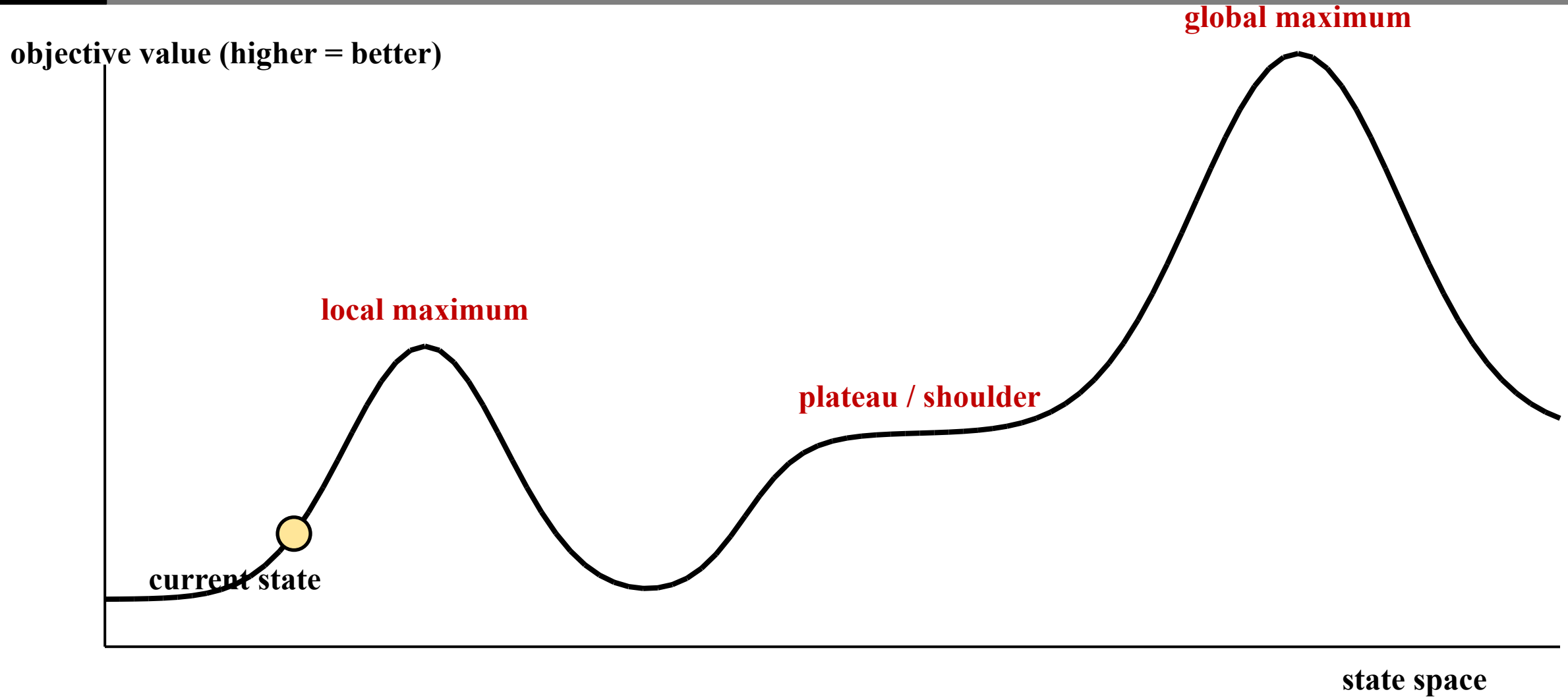
Algorithm	Expands lowest	Optimal?	Dhaka → Chattogram	Trap graph
Uniform-Cost	$g(n)$	Yes	8 expansions, 275 km	$S \rightarrow B \rightarrow G = 10$ ✓
Greedy Best-First	$h(n)$	No	5 expansions, 275 km	$S \rightarrow A \rightarrow G = 11$ ✗
A*	$g(n) + h(n)$	Yes (admissible h)	6 expansions, 275 km	$S \rightarrow B \rightarrow G = 10$ ✓

- ▶ Greedy: fast, but can be wrong.
- ▶ A*: optimal and usually much faster than UCS.
- ▶ Better admissible heuristic → fewer expansions.

Local Search: A Different Idea

- ▶ Sometimes we only need a good final state, not the path: 8-queens, class timetables, delivery routes.
- ▶ Local search keeps only one current state and moves to a neighbouring state.
- ▶ Uses very little memory — no frontier, no explored set.
- ▶ Works on huge or infinite state spaces.
- ▶ Not systematic: it can miss the best solution.
- ▶ The most basic local search: Hill Climbing.

The State-Space Landscape



Hill Climbing Algorithm

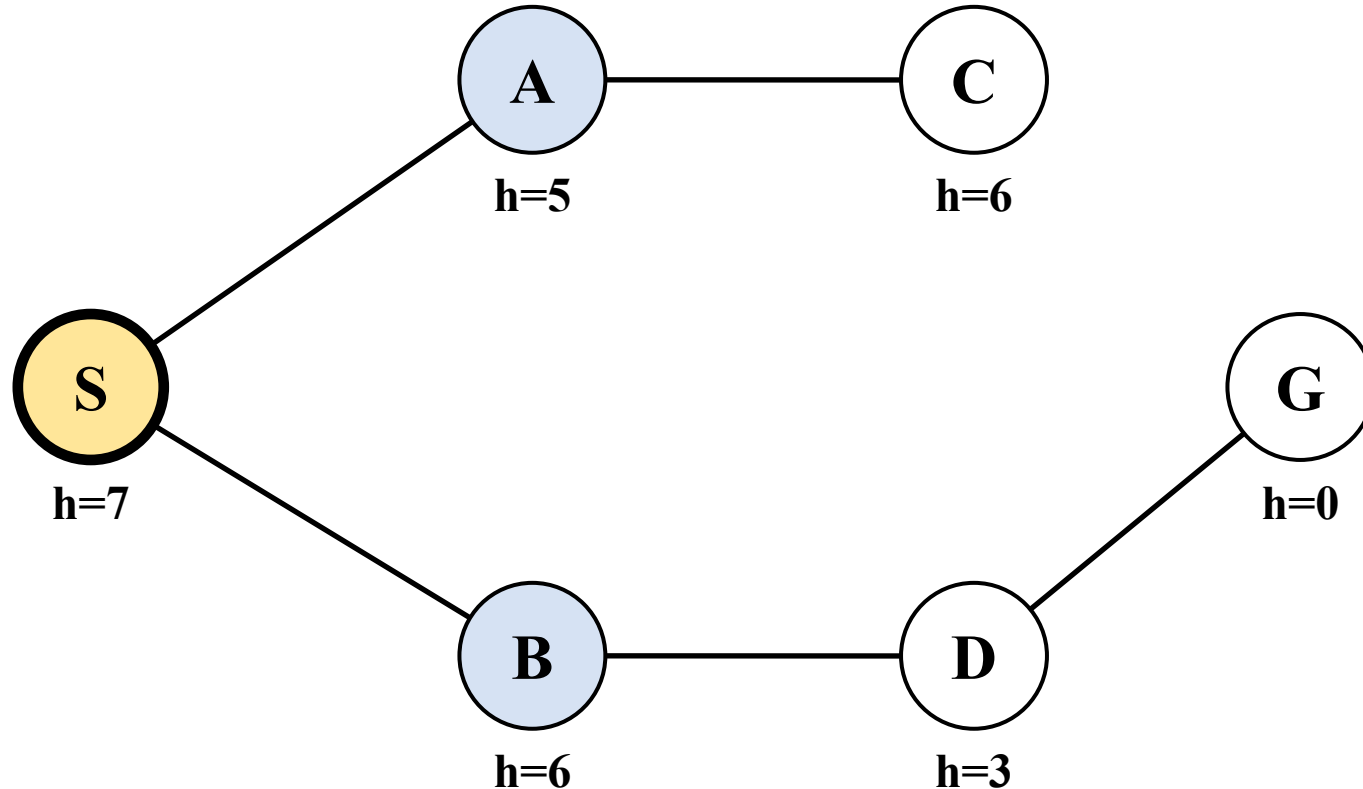
```
function HILL-CLIMBING(problem) returns a state
  current ← initial state
  loop do
    neighbour ← best successor of current
    if value(neighbour) ≤ value(current) then
      return current          // a peak (maybe local!)
    current ← neighbour
```

- ▶ Looks only at immediate neighbours — “climbing a hill in fog”.
- ▶ Never goes downhill, never backtracks.
- ▶ With a cost h (lower is better), move to the neighbour with the lowest h .

Hill Climbing: Step by Step (1/5)

Step 1: at S ($h = 7$)

Neighbours: A ($h = 5$), B ($h = 6$).
Best is A, and $5 < 7 \rightarrow$ move to A.



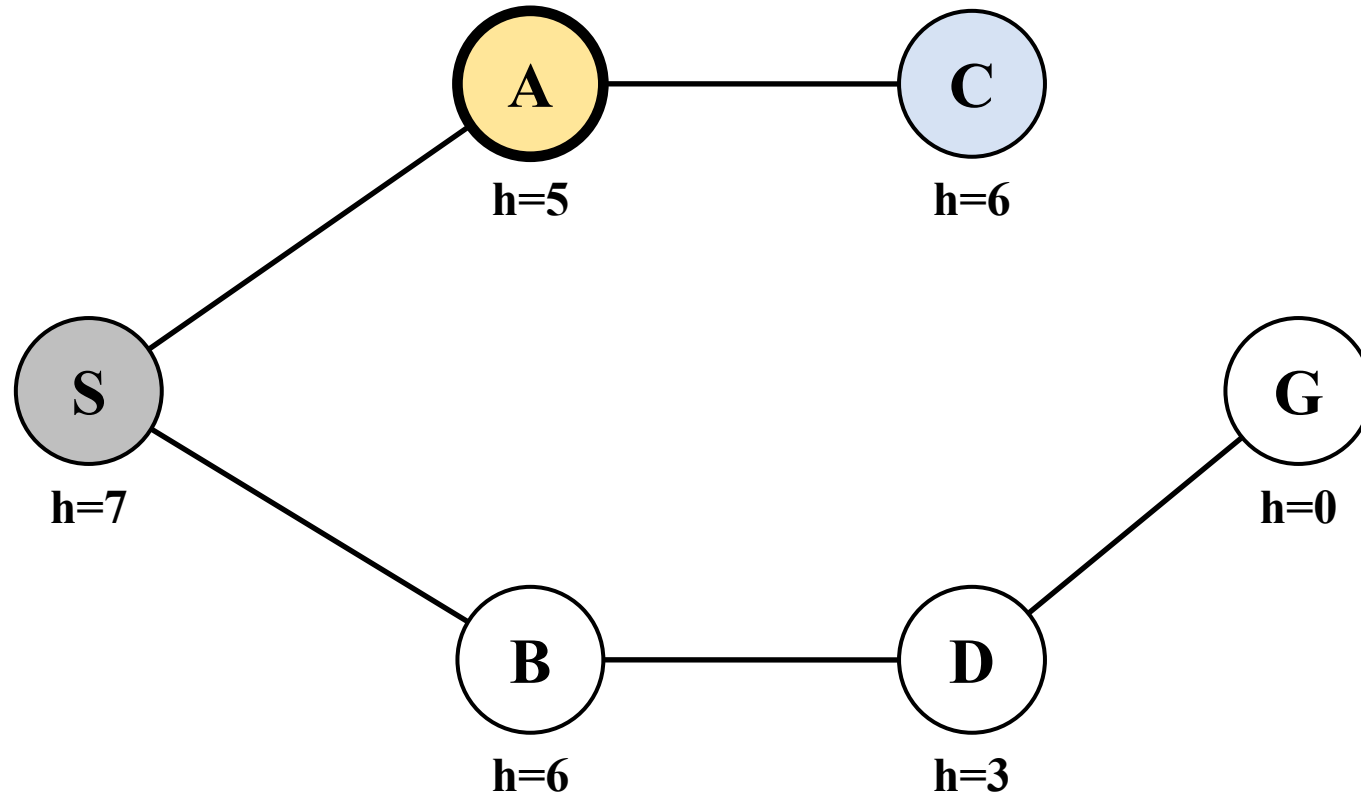
 Current

 Neighbour

 Visited

Lower h is better here (we walk downhill toward $h = 0$).

Hill Climbing: Step by Step (2/5)



Current

Neighbour

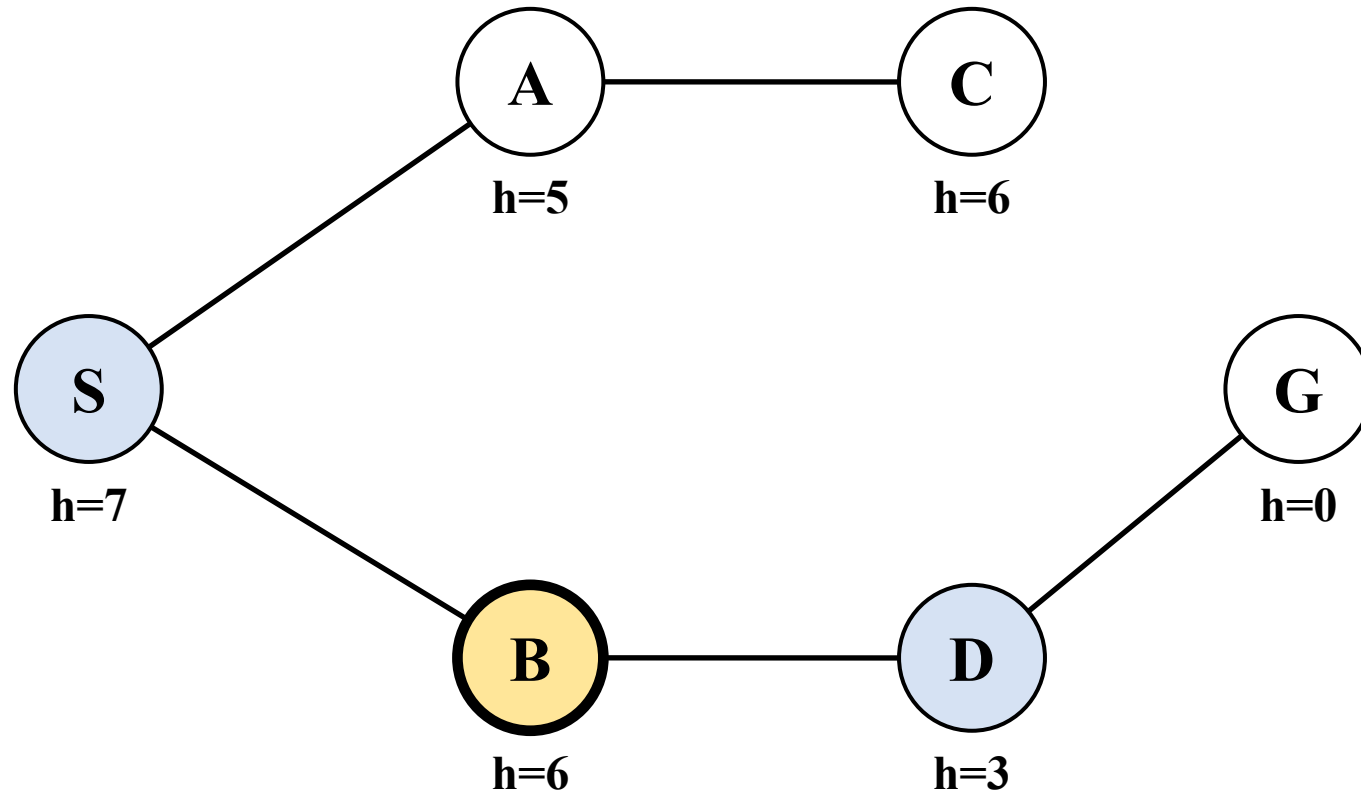
Visited

Step 2: at A ($h = 5$)

Neighbours: S ($h = 7$), C ($h = 6$).
Both are **WORSE** than 5 $\rightarrow$ stop.
Stuck at A: a local minimum! The goal G is never reached.

Lower h is better here (we walk downhill toward $h = 0$).

Hill Climbing: Step by Step (3/5)



 Current

 Neighbour

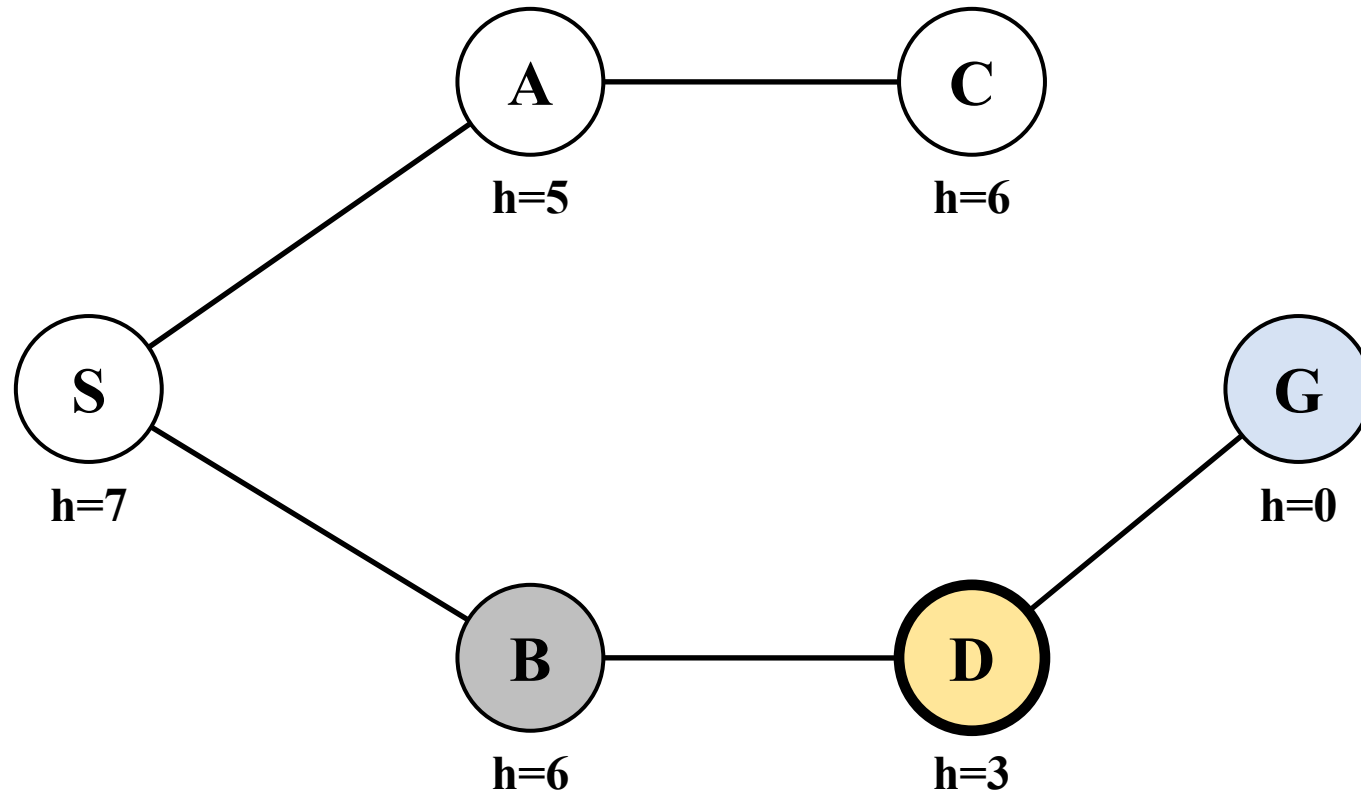
 Visited

Random restart at B ($h = 6$)

Neighbours: S (7), D (3). Move to D.

Lower h is better here (we walk downhill toward $h = 0$).

Hill Climbing: Step by Step (4/5)

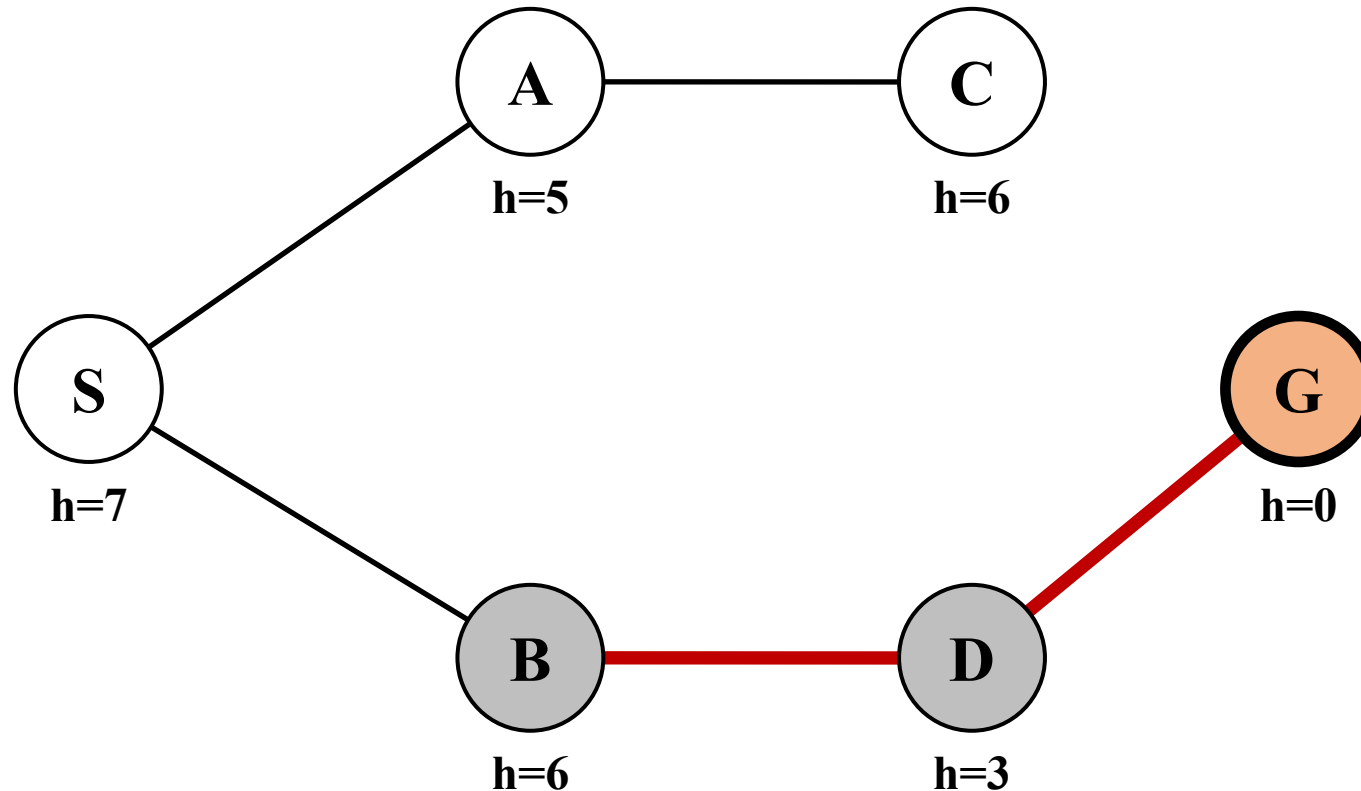


At D ($h = 3$)

Neighbours: B (6), G (0). Move to G.

Lower h is better here (we walk downhill toward $h = 0$).

Hill Climbing: Step by Step (5/5)



At G ($h = 0$) — GOAL ✓

**Restarting from another state
escaped the local minimum: $B \rightarrow D \rightarrow G$.**

**Lower h is better here (we walk downhill
toward $h = 0$).**

 **Current**

 **Neighbour**

 **Visited**

Example: 8-Queens with Hill Climbing

18	12	14	13	13	12	14	14
14	16	13	15	12	14	12	16
14	12	18	13	15	12	14	14
15	14	14	Q	13	16	13	16
Q	14	17	15	Q	14	16	16
17	Q	16	18	15	Q	15	Q
18	14	Q	15	15	14	Q	16
14	14	13	17	12	14	12	18

- ▶ Place 8 queens so that no two attack each other.
- ▶ State: one queen in each column.
- ▶ Neighbour: move one queen within its column ($8 \times 7 = 56$ neighbours).
- ▶ h = number of attacking pairs. Current board: $h = 17$
- ▶ Numbers = h after that move.
- ▶ Yellow squares = best moves ($h = 12$). Hill climbing picks one.

8-Queens: After One Step

	Q						
			Q				
Q				Q			
					Q		Q
		Q				Q	

- ▶ Queen in column 2 moved to the top row → $h = 12$
- ▶ Repeat until no move lowers h .
- ▶ Result on random 8-queens boards:
 - solved about 14% of the time
 - stuck about 86% of the time (local minimum)
- ▶ So we need ways to escape...

Why Hill Climbing Gets Stuck

Problem	What happens	Picture
Local maximum	A peak higher than its neighbours but lower than the best peak	Small hill next to a mountain
Plateau	A flat area: all neighbours have the same value → no direction	Flat field in fog
Ridge	A sequence of local maxima that single moves cannot climb	Narrow mountain ridge

- ▶ Hill climbing is not complete and not optimal.
- ▶ But it is fast and uses almost no memory.

Escaping: Variants of Hill Climbing

Variant	Idea
Random-restart	Run hill climbing many times from random starts; keep the best result
Sideways moves	Allow moves to equal-valued neighbours to cross a plateau (limit the number)
Stochastic	Choose randomly among the uphill moves
Simulated annealing	Sometimes accept a WORSE move, with probability $e^{\Delta E/T}$; T (temperature) slowly decreases

Temperature T	10	1	0.1
P(accept a move that is 2 worse)	0.82	0.14	≈ 0

Hill Climbing in Python (Lab Link)

```
def hill_climbing(start, neighbours, h):
    current = start
    while True:
        best = min(neighbours(current), key=h)
        if h(best) >= h(current):    # no better neighbour
            return current
        current = best

graph = {'S': ['A', 'B'], 'A': ['S', 'C'], 'B': ['S', 'D'],
        'C': ['A'], 'D': ['B', 'G'], 'G': ['D']}
h = {'S': 7, 'A': 5, 'B': 6, 'C': 6, 'D': 3, 'G': 0}

print(hill_climbing('S', graph.get, h.get))    # A    (stuck!)
print(hill_climbing('B', graph.get, h.get))    # G    (goal)
```

A* vs Hill Climbing

	A* (systematic)	Hill Climbing (local)
Keeps in memory	Frontier + explored set	Only the current state
Returns	A path to the goal	A good final state
Optimal?	Yes (admissible h)	No
Complete?	Yes	No — can get stuck
Memory	Large (exponential)	Very small (constant)
Good for	Route finding, puzzles	8-queens, scheduling, optimisation

Quick Check

- ▶ 1. Which algorithm expands the node with the lowest $h(n)$ only?
- ▶ 2. In A^* , Cumilla has $g = 100$ and $h = 140$. What is f ?
- ▶ 3. Is A^* always optimal, whatever h we use?
- ▶ 4. Why did hill climbing stop at A in our example?
- ▶ 5. Name one way to escape a local maximum.

Summary

- ▶ A heuristic $h(n)$ estimates the cost to the goal.
- ▶ Greedy expands lowest h : fast but not optimal.
- ▶ A^* expands lowest $g + h$: optimal with an admissible heuristic; better heuristics expand fewer nodes.
- ▶ Hill climbing keeps one state and moves to the best neighbour: little memory, but can get stuck (local maxima, plateaus, ridges).
- ▶ Random restart and simulated annealing help escape.
- ▶ Reading: Russell & Norvig, AIMA, 3.5–3.6 and 4.1.
- ▶ Next lecture: Problem Reduction — AND–OR graphs and AO*

Thank You

Any questions?